

Tag 3: Inventar-App

Ziel der Übung

Baue ausgehend von

- der App aus **Tag-2 mit MQTT** und
- dem **Grades-Starter** ([g_starter](#))

eine kleine **Inventar-App** (Department, Person, Device, Assignment), bei der

- das **DB-Schema** über ein **Init-SQL** im Projekt definiert wird,
- zentrale **Domänenregeln** (z. B. max. eine aktive Ausleihe) technisch umgesetzt sind,
- du beim Vorführen **Branch-Wechsel** und wichtige Teile der App **erklären** kannst.

Abgabe (mündliche Demo)

Ihr zeigt in der Gruppe:

1. Tag-2-Branch (MQTT)

- App läuft.
- MQTT-Chat zwischen zwei Terminals (oder zwei Personen) mit [mosquitto_pub](#) / [mosquitto_sub](#).

2. Branch-Wechsel

- [git checkout <tag2-branch>](#) → kurze MQTT-Demo.
- [git checkout <inventar-branch>](#) → Inventar-App starten.

3. Inventar-App

- Health ([/health](#)) und API-Doku ([/docs](#)) funktionieren.

- Inventar-Schema liegt als **Init-SQL** im Projekt (z. B. [db/init/010_inventory_schema.sql](#)).
- Mindestens:
 - * CRUD für Devices
 - * Erstellen und Auflisten von Assignments
 - * Rückgabe von Assignments
 - * Domänenregeln werden eingehalten (s. Abschnitt D).

4. Code erklären (kurz)

Jede Gruppe erklärt mindestens:

- eine Stelle im **Schema-SQL** (Tabellen + Schlüssel),
- einen **API-Endpunkt** (z. B. [POST /assignments](#)),
- einen **UI-Flow** (z. B. „Gerät ausgeben“ mit oder ohne htmx).

Voraussetzungen

- Repo mit Tag-2-Stand (Inventar + MQTT) vorhanden.
- [g_starter.zip](#) liegt vor (Grades-Beispiel).
- Docker & Compose, VS Code + Dev-Container Extension.
- Git-Remote (GitHub o. ä.) konfiguriert.

A) MQTT-Warm-up mit der Tag-2-App

Zeitbudget: ca. 30 Min.

1. **Branch von Tag-2 verwenden**
2. **Tag-2-App starten**

- `docker compose up -d`
- Prüfe:
 - * <http://localhost:8000/health>
 - * <http://localhost:8000/docs>

3. **MQTT-Chat im Terminal**

- **Terminal 1 (Subscriber):** starten
- **Terminal 2 (Publisher):** Nachricht senden
- Variiere:
 - * andere Topics (z. B. `lab/chat/gruppe2`)
 - * einfache JSON-Nachrichten, z.B.


```
{"from": "alice", "msg": "Ping"}
```

4. Mini-Reflexion (Stichworte genügen)

- Welche Bausteine spielen zusammen? (**Publisher** → **Broker** → **Subscriber**)
- Wo taucht MQTT in euren **Logs** oder im Code auf?

B) Grades-Starter verstehen

Zeitbudget: ca. 30 Min.

Ziel: Ihr nutzt `g_starter` als Blaupause für eure Inventar-App.

1. Grades-Starter separat öffnen

- `g_starter.zip` entpacken (z. B. in `grades-starter/`).
- Der-Container bauen und starten
- Prüfen:
 - * `http://localhost:8000/health`
 - * `http://localhost:8000/docs`
 - * UI (z. B. `/grades`, `/students`).

2. Struktur grob analysieren

- **DB-Init:** `db/init/001_schema.sql`
 - * Welche Tabellen gibt es (`student`, `module`, `grade`)?
 - * Wo stehen **Constraints** (UNIQUE, CHECK, FK)?
- **DB-Zugriff:** `backend/app/db.py`
 - * Wie wird `get_conn()` verwendet?
- **API:**

- * Wo werden Routen für Students / Grades definiert?
- **UI:**
 - * Templates unter [backend/app/templates/...](#) (Listen, Formulare).

3. Kurze Notizen machen (z. B. [docs/grades-notes.md](#))

- Tabellen + Beziehungen in Grades.
- Welche Teile wollt ihr für Inventar wiederverwenden?

C) Inventar-Branch für Tag 3 anlegen

Zeitbudget: ca. 15–20 min

1. Zurück in euer Inventar-Repo

- [git checkout main](#)
- Sicherstellen, dass [main](#) auf einem sauberen Stand ist (Tag-2-Ergebnis ist ok).

2. Neuer Branch für die Inventar-App

- (Optional) direkt pushen

3. Technische Basis klären

- Nutzt [g_starter](#) als Vorlage für:
- [db/init/](#)-Struktur,
- [backend/app](#)-Struktur,
- Docker-/Compose-Setup,
- MQTT-Konfiguration.
- **Klären: Wie genau kann es übernommen werden?**
(z. B. neues Repo vs. Umbau des vorhandenen Repos.)

D) Inventar-Schema als Init-SQL

Zeitbudget: ca. 20 min

Wichtig: **Keine Tabellen per Hand in der DB anlegen**, sondern über eine **Init-SQL-Datei im Projekt**.

1. Neue Schema-Datei anlegen

- Schema gemäß Definition aus Tag 2

2. Einfache Seed-Daten

- 2–3 Departments (z. B. „IT“, „Produktion“).
- 2–3 Device Types (z. B. „Laptop“, „Monitor“, „Handscanner“).
- ein paar **person**-, **location**- und **device**-Einträge.

3. Compose neu starten & Schema prüfen

- `docker compose down`
- `docker compose up -d`
- Mit `psql` im DB-Container nachsehen:

```
docker compose exec db psql -U appuser -d appdb -c "\dt"
docker compose exec db psql -U appuser -d appdb -c "\d device"
# oder mit
docker compose exec db psql -U appuser -d appdb
\dt
\d device
select * from device;
\q
```

- Ggf. Fehler im Init-SQL korrigieren und Container neu starten.

4. Commit

- Änderungen adden und committen

E) API & Domänenregeln für Inventar

Zeitbudget: ca. 45 min

Ziel: Minimaler, aber sauberer Flow: Geräte anlegen, ausgeben, zurückgeben – mit technischen Regeln.

1. API-Modelle anlegen

- Pydantic-Modelle

2. Endpunkte für Devices

- [GET /devices](#)
- Liste aller Geräte inkl. Typ, Standort, Status („frei“ oder „ausgeliehen“).
- [POST /devices](#)
- Neues Gerät mit [serial_number](#), [device_type_id](#), [location_id](#) (+ optional [note](#)).
- **Domänenregel IR-01 – Eindeutige Seriennummer**
- Wenn [serial_number](#) schon existiert:
 - * HTTP **409 Conflict** mit verständlicher Fehlermeldung im JSON.

3. Endpunkte für Assignments

- [GET /assignments/active](#)
- Liste aller laufenden Ausleihen ([returned_at IS NULL](#)).
- [POST /assignments](#)
- Gerät ausgeben:
 - * [device_id](#), [person_id](#), [issued_at](#).
- **Domänenregel IR-02 – Max. eine aktive Ausleihe pro Gerät**
- * Prüfe vor dem Insert, ob es für dieses [device_id](#) bereits ein Assignment mit [returned_at IS NULL](#) gibt.
- * Falls ja → HTTP **409**.
- [POST /assignments/{assignment_id}/return](#)
- Rückgabe:
 - * setzt [returned_at](#) auf „jetzt“ oder einen übergebenen Zeitpunkt.
- **Domänenregel IR-03 – Zeitlogik**
- * Nur zulässig, wenn [returned_at](#) $\geq$ [issued_at](#), sonst 422 / Fehlermeldung.

4. Tests über [/docs](#)

- Für jeden Endpunkt mind.:
- **Happy Path** (erwarteter Erfolg).
- mind. **einen Fehlerfall** pro Regel (IR-01 bis IR-03).
- Testfälle kurz notieren (Stichworte reichen).

5. Commit

- Code + evtl. neue Dateien adden:

F) Einfache Inventar-UI

Zeitbudget: ca. 45 min

1. Inventar-Übersichtsseite

- Route z.B. [/inventory](#).
- Template mit:
- Tabelle aller Devices (Serial, Typ, Standort, Status).
- Hinweis, ob das Gerät „frei“ oder „ausgeliehen“ ist.

2. Formular „Gerät hinzufügen“

- Einfaches HTML-Formular oder mit htmx:
- Eingaben: [serial_number](#), [device_type_id](#), [location_id](#).
- POST auf euren [/devices](#)-Endpunkt.
- Bei Fehlern (z. B. doppelte Seriennummer) Fehlermeldung anzeigen.

3. (Optional) Formular „Gerät ausgeben / zurückgeben“

- Dropdown für [person](#) und [device](#) (nur verfügbare Devices).
- Button „Ausgeben“ → [POST /assignments](#).
- In der Liste der aktiven Ausleihen ein Button „Zurückgeben“ → [POST /assignments/{id}/return](#).

4. Commit

- Template- und Routing-Änderungen committen

G) MQTT-Events für Ausleihen

Zeitbudget: ca. 20 min

1. Events im API-Code publizieren

- Beim erfolgreichen `POST /assignments`:
- Topic z. B. `inventory/assignments/issued`
- Payload (JSON) mit `device_id`, `person_id`, `issued_at`.
- Beim erfolgreichen Rückgabe-Call:
- Topic z. B. `inventory/assignments/returned`

2. Test mit `mosquitto_sub`

- Subscriber:

```
mosquitto_sub -h localhost -t 'inventory/assignments/#' -v
```

- In der UI eine Ausleihe auslösen und prüfen, welche Nachrichten ankommen.

3. Commit

H) Prüfliste & Präsentation

Prüfliste (Ende der Übung)

- [] Tag-2-Branch (`tag2-mqtt-demo` o. ä.) existiert und läuft mit MQTT-Chat.
- [] Inventar-Branch (`feat/inventar-app` o. ä.) existiert.
- [] Init-SQL für Inventar ist angelegt und erzeugt alle Tabellen.
- [] REST-Endpunkte für Devices & Assignments funktionieren inkl. Domänenregeln.
- [] Mindestens eine einfache UI-Seite für Inventar ist vorhanden.
- [] Ihr könnt Branch-Wechsel, Schema, API und einen UI-Flow **mündlich erklären**.

Ablauf in der Präsentation (Vorschlag)

1. `git checkout <tag2-branch>` → kurz MQTT-Chat demonstrieren.
2. `git checkout <inventar-branch>` → Inventar-App starten.
3. **Flow vorführen:** Gerät anlegen → ausgeben → als zurückgegeben markieren.
4. Anhand von 2–3 Code-Stellen erklären, **wie** eure Regeln technisch umgesetzt sind.